# JavaScript Course - YouLearn

**Audience:** Strong in Python, HTML, CSS, SQL, Flask/FastAPI
**Goal:** Become productive in JavaScript for **dynamic content**, **API-driven/headless frontends**, and modern tooling/frameworks (React, Vue, Node.js).

## Course Philosophy (How to use this)

Each section includes:

- **Goal** (what you should be able to do)
- **Core ideas** (what matters in real projects)
- **Python → JavaScript translation** (bridge from what you already know)
- **Build steps** (do this in order)
- **Common mistakes** (so you don't get stuck)
- **Checkpoint** (tiny deliverable)
- **Practical questions** (to confirm understanding)

## 1) The JavaScript Mental Model (Browser-first, Event-driven, Async)

### Goal

Understand why JavaScript behaves differently from Python when building UIs and calling APIs.

### Core ideas

- Browser JS is **event-driven** (runs on clicks, input, page load, request completion).
- UI code must avoid blocking: network calls are asynchronous.
- Most frontend "mystery bugs" are actually **timing/state** issues.

### Python → JavaScript translation

- Python scripts often run top-to-bottom.
- JavaScript in the browser reacts to events and uses `async/await` heavily.

### Build steps

1. Open DevTools Console.
2. Create a button in HTML.
3. Attach a click handler and log something.
4. Add an async function that waits for a fetch, then logs data.

### Common mistakes

- Expecting values from an async function immediately.
- Writing fetch logic outside event handlers and wondering why nothing runs.

### Checkpoint

Explain in your own words: “Why does an API response arrive *later* than the line after `fetch()`?”

### Practical questions

- What is the difference between **running code** and **registering a callback**?

---

## 2) Modern JavaScript Essentials (ES6+ you will actually use)

### Goal

Write clean modern JS that you’ll see in React/Vue and real codebases.

### Core ideas

- `const` by default, `let` only when reassignment is needed.
- Arrays + objects are the backbone of JSON and state.
- Use array methods for transformations.

### Python → JavaScript translation

- Python f-strings → template literals
- Python dicts → JS objects
- Python list comprehensions → `.map()`, `.filter()`, `.reduce()`

### Build steps (copy/paste & experiment)

```js
```

```
// Variables
const apiBase = "http://localhost:8000";
let tasks = [];

// Template literal
const url = `${apiBase}/tasks`;

// Arrow function
const toTitle = (t) => t.title;

// Destructuring
const task = { id: 1, title: "Learn JS", done: false };
const { id, title } = task;

// Array transforms
const titles = tasks.map(toTitle);
const undone = tasks.filter(t => !t.done);
```

### Common mistakes

- Using `var` (confusing scoping).
- Confusing `==` with `===` (prefer `===`).
- Mutating arrays/objects unintentionally.

### Checkpoint

Given an array of tasks, output a new array of titles for unfinished tasks using `.filter()` + `.map()`.

### Practical questions

- Why is `const` safer than `let` for most variables?

---

## 3) DOM & Dynamic Content (Vanilla JS UI Building)

### Goal

Update a page dynamically without reloads (the skill frameworks build on).

### Core ideas

- DOM = the live structure of the page.
- Rendering = turning state (JS data) into UI (DOM nodes).
- Events connect user actions to state updates.

## Python → JavaScript translation

- Server-rendered templates produce HTML once.
- In JS, you re-render parts of the UI when state changes.

## Build steps

1. Create a list container in HTML.
2. Store tasks in an array.
3. Write `renderTasks(tasks)` that rebuilds the list.
4. Add a form that pushes a task into the array and calls render again.

html

```html
<ul id="taskList"></ul>
<form id="taskForm">
  <input id="taskInput" />
  <button type="submit">Add</button>
</form>
```

js

```
const taskListEl = document.querySelector("#taskList");
const formEl = document.querySelector("#taskForm");
const inputEl = document.querySelector("#taskInput");

let tasks = [];

function renderTasks(tasks) {
  taskListEl.innerHTML = "";
  for (const t of tasks) {
    const li = document.createElement("li");
    li.textContent = t.title;
    taskListEl.appendChild(li);
  }
}

formEl.addEventListener("submit", (e) => {
  e.preventDefault();
  tasks.push({ id: Date.now(), title: inputEl.value, done: false });
  inputEl.value = "";
  renderTasks(tasks);
});
```

### Common mistakes

- Forgetting `e.preventDefault()` on forms (page refresh).
- Adding event listeners repeatedly inside `renderTasks`.
- Using `innerHTML` with untrusted text (XSS risk).

### Checkpoint

Add a “Delete” button per item and remove it from the array by `id`.

### Practical questions

- What is “state” in a frontend?
- What’s a safe way to insert user text into the DOM?

---

## 4) HTTP + API Integration (Fetch, JSON, Errors, Auth, CORS)

### Goal

Build headless-style apps: frontend talks directly to your API layer (FastAPI) with proper error handling.

## Core ideas

- Every request has: method, URL, headers, body.
- Always handle:
  - non-200 responses (`res.ok`)
- JSON parsing
- loading and error UI

## Python → JavaScript translation

- Python `requests.get()` → JS `fetch()`
- Python `response.json()` → JS `await res.json()`

## Build steps

### GET

```js
async function fetchTasks() {
  const res = await fetch("http://localhost:8000/tasks");
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return await res.json();
}
```

### POST

```js
async function createTask(title) {
  const res = await fetch("http://localhost:8000/tasks", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title })
  });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return await res.json();
}
```

### Auth header pattern

```js
```

```
async function fetchWithAuth(url, token) {
  const res = await fetch(url, {
    headers: { "Authorization": `Bearer ${token}` }
  });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return await res.json();
}
```

### Common mistakes

- Forgetting `"Content-Type": "application/json"` on POST/PUT.
- Assuming errors always return JSON.
- CORS errors: trying to fix them in frontend instead of allowing origins in the API.

### Checkpoint

Integrate your DOM app so:

- On load, it fetches tasks from the API and renders.
- On submit, it POSTs a task, then refreshes the list.

### Practical questions

- What does CORS protect against?
- Where can you confirm a request actually sent headers correctly?

---

## 5) Frontend Architecture (How to avoid “one huge app.js”)

### Goal

Organize a frontend like you organize FastAPI projects: clean separation of concerns.

### Core ideas

Split into layers:

- **API layer**: request functions only
- **State layer**: in-memory data + helpers
- **UI layer**: render functions
- **Events layer**: event listeners and handlers
- **App entry**: bootstrapping

### Suggested structure

- `api.js`
- `state.js`
- `ui.js`
- `events.js`
- `app.js`

### Common mistakes

- Mixing DOM updates inside API functions.
- Global variables modified from everywhere.

### Checkpoint

Refactor your working app into `api.js` + `ui.js` at minimum.

### Practical questions

- What breaks first when everything is in one file?

---

## 6) React Core (Components, Props, State, Hooks)

### Goal

Build the same UI with React's state-driven rendering model.

### Core ideas

- Component = function that returns UI.
- State drives rerenders.
- Effects handle side effects like fetching.

### Build steps (minimum concepts)

- Create app with Vite.
- Make `TaskApp` component.
- Render list from state.
- Add form controlled by state.

```jsx
```

```
import { useEffect, useState } from "react";

export default function TaskApp() {
  const [tasks, setTasks] = useState([]);

  return (
    <div>
      <h1>Tasks</h1>
      {tasks.map(t => <div key={t.id}>{t.title}</div>)}
    </div>
  );
}
```

### Common mistakes

- Mutating arrays directly instead of creating a new array.
- Missing `key` in list rendering.
- Fetching in render instead of `useEffect`.

### Checkpoint

Render tasks from a hardcoded array using `.map()` and correct keys.

### Practical questions

- Why does mutating state directly often fail to update the UI?

---

## 7) React + APIs (CRUD, Loading/Error UX, Routing Basics)

### Goal

Create a React frontend that does real CRUD against your API with good UX.

### Core ideas

- Loading state, error state, empty state.
- Either refetch after POST/DELETE or update state locally.
- Routing for multi-page feel (React Router).

### Build steps

1. Add `isLoading` and `error`.
2. Fetch tasks on mount.

3. Build create + delete.
4. Add routing if you want multiple pages.

**Fetch on mount**

```jsx
useEffect(() => {
  let ignore = false;

  (async () => {
    try {
      const res = await fetch("http://localhost:8000/tasks");
      const data = await res.json();
      if (!ignore) setTasks(data);
    } catch (e) {
      // set error state
    }
  })();

  return () => { ignore = true; };
}, []);
```

### Common mistakes

- Infinite loops from incorrect effect dependencies.
- Not checking response status.
- Not showing loading/error UI, making the app feel broken.

### Checkpoint

Implement create task:

- submit form → POST → update list (refetch or state update)

### Practical questions

- When would you refetch instead of updating state locally?

---

## 8) Vue Quick Translation (So you can read Vue and choose later)

### Goal

Understand how Vue expresses the same concepts as React.

### Core ideas

- Reactive state via refs
- Template syntax vs JSX
- Lifecycle hooks like `onMounted`

### Checkpoint

Read a Vue component and identify:

- state
- API call location
- rendering loop

---

## 9) Node.js + Express Basics (Backend JS literacy)

### Goal

Be able to write and understand simple Node services, scripts, and tooling.

### Core ideas

- Node runs JS outside the browser.
- Express is a minimal server framework (conceptually like Flask).
- Async I/O is central.

### Build steps

1. Create a tiny Express server.
2. Add `/health` route returning JSON.
3. Add one CRUD route and log requests.

### Common mistakes

- Treating Node like multi-threaded Python by default.
- Forgetting async error handling.

### Checkpoint

Make `/health` return `{ status: "ok" }` and confirm in browser.

### Practical questions

- Why is Node commonly used even in teams that use Python backends?

---

## 10) Capstone: Headless Task App (Vanilla → React)

### Goal

Prove competence by building one complete small system.

### Requirements

- FastAPI backend (you already have this skill)
- Frontend:
  - Vanilla JS version: DOM + Fetch CRUD
- React version: hooks + CRUD + loading/error UX

### Deliverables

- A working UI that:
  - lists tasks
- creates tasks
- deletes tasks

A README with:

- how to run backend
- how to run frontend
- what endpoints are used

### “Professional polish” (optional)

- Input validation and user feedback
- Debounced search box
- Pagination or filtering
- Deploy frontend separately from API (headless deployment)

---